Date

Expt. No. 3. (A) Page No. 13

Aim:- To compute topological ordering of DAG using DFS-method based ~~method~~ (or Kahn's algorithm)

Theory:- Topological sort orders vertices of a DAG such that for every directed edge directed edge u->v, u comes before v. Can be done via DFS post-order or Kahn's algorithm using indegrees.

Algorithm:-

Topological_Sort (G,V)

1. Calculate indegree of every vertex.
2. Create an empty queue Q.
3. Create an empty list ORDER.
4. For each vertex v in V:
   If indegree[v] == 0:
   Enqueue (Q,v)

5. While Q is not empty:
   u = Dequeue(Q)
   Add u to ORDER

   FOR each vertex v adjacent to u:
   indegree[v] = indegree[v] - 1

   If indegree[v] == 0;
   Enqueue (Q,v)

6. If size(ORDER == number of vertices:
   Print ORDER
   Else: Print "Topological ordering not possible"
7. Stop.

Teacher's Signature

C++ Program :-

```
#include <bits/stdc++.h>
using namespace std;

void dfsTopo(int v, vector<vector<int>> &adj,
    vector<int> &vis, vector<int> &order) {
      vis[v] = 1;
      for(int u : adj[v]) if(!vis[u]) dfsTopo(u, adj,
                              vis, order);
      order.push_back(v);
    }
int main() {
    int V = 6;
    vector<vector<int>> adj(V);
    adj[5] = {2,0};
    adj[4] = {0,1};
    adj[2] = {3};
    adj[3] = {1};
  vector<int> vis(V,0), order;
  for(int i=0; i<V; i++)
    { if(!vis[i])
        dfsTopo(i, adj, vis, order);
      }
    reverse(order.begin(), order.end());
    for(int x : order) cout<<x<<" ";
    return 0;
}
```

Teacher's Signature

Time Complexity O(V+E)
Space Complexity : O(V)

Viva Question :-
3 (B)

Aim :- Compute the transitive closure of a directed graph using Warshall's algorithm.

Algorithm :-

WARSHALL (A, n)
1. T = A
2. FOR k = 1 TO n
3. FOR i = 1 TO n
4. FOR j = 1 TO n
5. T[i][j] = T[i][j] OR (T[i][k] AND T[k][j])
6. END FOR
7. END FOR
8. END FOR
9. PRINT T
10. STOP

C++ Program

```
#include <bits/stdc++.h>
using namespace std;
int main(){
  int n=4;
```

```
vector<vector<int>> reach = {
        {1, 1, 0, 0},
        {0, 1, 1, 0},
        {0, 0, 1, 1},
        {0, 0, 0, 1}
    };

for(int k = 0; k < n; k++)
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            reach[i][j] = reach[i][j] || (reach[i][k]
                && reach[k][j]);
    for(auto &row : reach) {
        for(int v : row) cout << v << ' ';
        cout << "\n";
    }
    return 0;
}
```

Complexity

Time Complexity = $O(n^3)$

Space complexity = $O(n^2)$

Teacher's Signature _____

Date

Expt. No. Page No. 18.

Viva Questions

Q1 What is transitive closure?

Ans It tells whether a path exists between every pair of vertices in a directed graph.

Q2 What is Warshall's algorithm used for?

Ans It is used to find the transitive closure of a directed graph.

Q3 What is the time complexity of Warshall's algorithm?

Ans Its time complexity is $O(n^3)$.

Q4 What is topological ordering of a directed Graph?

Ans It is a linear ordering of vertices such that for every edge $u \rightarrow v$, vertex u comes before v.

Q5 Can we perform topological sorting on a cyclic graph?

Ans NO, topological sorting is possible only for a Directed Acyclic Graph (DAG).

Teacher's Signature

Date

Expt. No. 4. Page No. 19.

Aim:- Implement 0/1 knapsack problem using Dynamic Programming.

Theory:- The 0/1 knapsack is an optimization problem solved using Dynamic programming. We are given n items, each having a weight & a profit & a knapsack with fixed capacity W.

Algorithm

0/1 knapsack(profit, weight, n, W)

1. Create DP[n+1][W+1]
2. FOR i = 0 To n
   FOR w = 0 TO W
   If i == 0 OR W == 0
   DP[i][w] = 0
   ELSE If weight[i] <= W
   DP[i][W] = MAX(
   DP[i-1][w],
   profit[i] + DP[i-1][w - weight[i]])

   ELSE
   DP[i][w] = DP[i-1][w]

3. RETURN DP[n][W]
4. STOP.

Teacher's Signature

Expt. No. _____ Date _____

Page No. 20

C++ Program

```
#include <bits/stdc++.h>
using namespace std;

int knapsack(int W, vector<int> &wt,
             vector<int> &val)
{
  int n = wt.size();
  vector<vector<int>> dp(n+1, vector<int>
                         (W+1,0));
  for(int i=1; i<=n; i++)
    {
      for(int w=0; w<=W; w++) {
        if(wt[i-1]<=w)
    {
      dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]]
                     , dp[i-1][w]); }
    else dp[i][w] = dp[i-1][w];
    }
  }
  return dp[n][W];
}
int main(){
  vector<int> val = {60, 100, 120};
  vector<int> wt = {10, 20, 30};
  int W = 50;
  cout<< knapsack(W, wt, val)<< "\n";
  return 0;
}
```

Teacher's Signature _____

Date

Expt. No. Page No. 21

Complexity

Time Complexity = O(nW)
Space Complexity = O(nW)

Here n = number of items.
W = knapsack capacity

Viva Questions

Q1 What is the 0/1 knapsack problem?
Ans It is an optimization problem in which we select items to maximize profit without exceeding the knapsack capacity.

Q2 Why is it called 0/1 knapsack problem?
Ans Because each item is either selected completely (1) or not selected (0).

Q3 Which technique is used to solve this problem?
Ans Dynamic programming is used to solve it by storing the results of smaller subproblems.

Q4 What is the time complexity of 0/1 knapsack using D.P.?
Ans The time complexity is O(nW), where n is number of items & W is knapsack capacity

Q5 Can we select an item partially in 0/1 knapsack?
Ans No, partial selection is not allowed. An item must be selected completely or not selected at all.

Teacher's Signature